

Ceng 382 Web Project

Full-Stack Catering Website Project

Contents

Ceng 382 Web Project.....	1
Full-Stack Catering Website Project	1
Objective.....	3
General Scenario	3
Required User Roles	3
Functional Requirements	3
2. Role-Based Dashboards.....	3
3. Menu Table.....	4
4. Google Maps API Usage.....	6
5. Rating and Comment System	6
6. Email System	6
7. Live Call System	6
9. Logging System	7
12. Cart and Payment System	8
13. Unique Design and Branding	9
Technical Expectations	10
Submission Requirements.....	12
Grading Rubric.....	13
Main Project Evaluation (100 Points)	13
Bonus Evaluation (Up to +30 Points).....	14

Objective

In this project, students are expected to design and implement a **full-stack catering website** with role-based access, authentication, order management, location-based restaurant filtering, payment simulation, logging, and dynamically generated documents.

Students are expected to build a **unique, branded website, not a plain**, template-based copy. The project must demonstrate both technical functionality and design effort.

General Scenario

You are asked to develop a catering platform where users can browse nearby restaurants, place food orders, interact with caterers, and track their purchases. The system must support different roles, logging, order-related communication, payment simulation, and document generation.

Required User Roles

Your system must include the following **three roles**:

1. **Admin**
2. **Caterers**
3. **Users**

Each role must have appropriate permissions and separate access logic where necessary.

Functional Requirements

1. Authentication and Authorization

- The website must include a **working login system**.
- Authentication and authorization must be implemented properly.
- Users must only access pages and actions allowed for their role.
- Registration may be included if needed for your design.

Bonus

- **Two-Factor Authentication with Email: +10 bonus points**

2. Role-Based Dashboards

After logging in, each role must see a meaningful dashboard.

Admin Dashboard

The admin panel should allow the administrator to:

- monitor system activity,
- access logs,
- inspect users/caterers/orders,
- manage data where necessary.

Caterer Dashboard

Caterers should see statistics related to their activity, such as:

- total received orders,
- completed orders,
- ratings,
- customer-related summaries,
- earnings or revenue simulation if implemented.

User Dashboard

Users should see statistics related to their own usage, such as:

- total purchases,
- order history,
- consumption statistics,
- total amount spent,
- recent activity.

3. Menu Table

The system must include a **menu table/page** where users can:

- browse available food items,
- choose what to buy,
- see restaurant-related information.

Caterer Responsibilities

Caterers must be able to **create and manage menu items**.

While creating a menu item, caterers must:

- add a **title/name**,
- define a **price**,
- provide a **description**,
- upload an **image for the menu item**.

In addition, caterers must define **customization options** for each menu item.

Menu Customization Definition (By Caterer)

For each menu item, caterers must be able to define:

- removable ingredients (e.g., no onion, no tomato),
- optional additions (e.g., extra cheese, extra meat),
- option groups (e.g., sauces, sides),
- whether an option changes the price.

Customization must:

- be **configurable per menu item**,
- not be hardcoded,
- be stored and managed in the system dynamically.

User View Requirements

- Users must be able to see:
 - menu item details,
 - associated caterer/restaurant,
 - **average rating of the menu item**,
 - **average rating of the caterer**.

Both rating values must be calculated dynamically based on previous completed orders.

Rating System Rules

- Users must be able to:
 - rate the **menu item** they purchased,
 - rate the **caterer** separately,
 - leave a **comment** tied to a specific order.

Ratings must:

- be linked to completed orders only,
- update average scores automatically,
- be displayed in the menu and caterer views.

Important Requirement

- Users must only see **restaurants that are close to them**.

4. Google Maps API Usage

For **all map actions** and **all location-based boolean checks**, students must use the **Google Maps API**.

Examples include:

- determining whether a restaurant is close to the user,
- distance-based filtering,
- location matching,
- map visualization,
- route or positional checks if included.

Using another map library for the main required location logic is not acceptable for this project.

5. Rating and Comment System

The system must include a **rating system** where:

- users can rate caterers,
- users can write comments,
- ratings/comments must be associated with a **specific order**.

This means ratings must not be random profile comments; they must be tied to an actual completed purchase.

6. Email System

A working email system must be implemented.

After a purchase:

- the user must receive an email,
- The caterer must receive an email.

The email should include meaningful purchase details, such as:

- order ID,
- customer/caterer information,
- ordered items,
- total price,
- purchase time,
- order status or summary.

7. Live Call System

After the purchase is completed, the system may allow live communication between both sides.

Big Bonus

- **Live call system between both sides after purchasing: +20 bonus points**

This feature is optional but highly rewarded.

8. Receipt Form and Agreement PDF Generation

For every order, the system must generate:

- a **receipt/form**,
- an **agreement PDF**.

Important Rule

These documents **must not be static files**.

They must be **generated dynamically with code**.

This means:

- document contents must change according to the order/customer/caterer data,
- PDF generation must be programmatic,
- order-specific values must appear automatically.

Each caterer should use these generated agreement documents for their customers.

9. Logging System

The website must include a logging system that records important events.

Examples include:

- login attempts,
- successful logins,
- failed logins,
- order creation,
- order updates,
- payment actions,
- rating submissions,
- email sending events,
- admin actions,
- errors or exceptions.

Students are expected to decide on a reasonable logging structure and store logs properly.

10. Admin Logging Page

The admin must have access to a **logging page** where logs can be viewed.

This page should allow the admin to inspect what has happened in the system.

11. Filterable and Paginated Tables

Every table on the website must:

- support **filtering**,
- already be in **pagination form**.

This applies to all major tables, such as:

- menu tables,
- users,
- caterers,
- orders,
- logs,
- ratings/comments,
- or any other data tables you include.

12. Cart and Payment System

The system must include a fully functional **cart and payment flow**.

Cart Requirements

Users must be able to:

- add menu items to cart using an **“Add to Cart”** action,
- select customization options defined by the caterer,
- update quantities,
- remove items,
- view total price dynamically.

The cart must display:

- selected items,
- quantities,
- selected customization options,
- item-based subtotal,

- total order amount.

Payment Screen

After finalizing the cart, users must proceed to a **payment screen**.

The payment screen must:

- collect card information,
- display full order summary,
- include customization details,
- show total price,
- simulate payment confirmation.

Important Note

This is a **simulation only**.

Real card usage is strictly prohibited.

After Payment

A successful payment must trigger:

- order creation,
- receipt generation,
- agreement PDF generation,
- email notifications,
- logging.

13. Unique Design and Branding

Each website must be **unique**.

Students must:

- create their own brand identity,
- style the site with CSS,
- avoid default/plain/unmodified templates,
- show visual effort in layout, color palette, typography, and overall design.

Identical or heavily copied designs may result in grade reduction.

Technical Expectations

Students are expected to demonstrate a solid understanding of **full-stack application development** and software engineering principles.

Your project must include:

Backend Development

- Clean and structured backend architecture,
- Proper separation of concerns (e.g., controller/service/repository or equivalent),
- Role-based authorization logic,
- Secure authentication handling,
- Proper error handling and validation.

Database Design

- Well-designed relational database schema,
- Proper use of relationships (e.g., one-to-many, many-to-many),
- Support for:
 - users,
 - caterers,
 - menu items,
 - customization options,
 - orders,
 - ratings (menu + caterer),
 - logs.
- Avoid unnecessary redundancy and ensure normalization where appropriate.

Frontend Development

- Clean and user-friendly UI,
- Proper page structure and navigation,
- Responsive or at least well-aligned layout,
- Clear display of data (tables, dashboards, cart, etc.).

API & External Services

- Correct and meaningful usage of **Google Maps API** for:
 - location-based filtering,

- distance checks,
 - map-related logic.
- Proper integration of an **email service**.

Business Logic Implementation

- Correct order flow (menu → cart → payment → order),
- Dynamic price calculation (including customization),
- Role-based dashboards with meaningful statistics,
- Rating system tied to completed orders.

Dynamic Content Generation

- Receipt/form generation per order,
- Agreement PDF generation using code (not static files).

Logging and Monitoring

- A structured logging system capturing system activity,
- Ability for admin to view logs through a UI.

Data Handling Features

- Filtering support in all tables,
- Pagination implemented across all large datasets.

Code Quality

- Readable and maintainable code,
- Meaningful naming conventions,
- Modular structure,
- Avoidance of hardcoded logic where dynamic structure is required.

Project Completeness

- All required features must be working,
- System must be demo-ready,
- No fake/mock flows for core functionalities.

Submission Requirements

Your submission must include all of the following:

1. **Project Report**

- A written report explaining your project
- It should describe your system design, architecture, database structure, technologies used, and implemented features

2. **Live Demo**

- A **maximum 10-minute live demo**
- You must demonstrate the major features of your system clearly and efficiently

3. **Database Generation Scripts**

- SQL scripts or equivalent database initialization scripts must be included
- The system must be reproducible

4. **References**

- You must provide the references/resources you used during development

Grading Rubric

Main Project Evaluation (100 Points)

Component	Points
Role structure (Admin, Caterers, Users) and authorization logic	10
Login authentication system	10
Role-based dashboards with meaningful statistics	10
Menu system (creation, image upload, customization definitions)	12
Cart system and order flow	8
Google Maps API integration (location filtering & checks)	10
Rating system (menu + caterer, order-linked)	8
Email system (both sides notified)	6
Dynamic receipt generation	5
Dynamic agreement PDF generation	5
Logging system (system-wide logging)	6
Admin logging page	4
Filterable & paginated tables	3
Payment simulation system	3
UI/UX design and unique branding	5
Report, database scripts, references, demo quality	5

Subtotal: 100 points

Bonus Evaluation (Up to +30 Points)

Bonus Feature	Points
Two-Factor Authentication (Email-based)	+10
Live Call System after purchase	+20

Maximum Total Score: 130 Points